

LLM Evaluation and Data Quality

A portfolio case study in response evaluation, function-calling quality, and enterprise task design.

Overview

This project examines how to evaluate language-model outputs and design tasks that can be graded consistently. It combines independent response scoring, tool-dependency analysis, and evidence-based review of enterprise workflows.

The central approach is to extract atomic requirements, ground each judgment in observable evidence, and distinguish critical failures from smaller quality issues. The resulting assessments connect model behavior to explicit criteria and actionable recommendations.

This case study presents the completed analysis. Scores are the assessments recorded in the report; they are not aggregate benchmark results or measurements from a new model run.

Project scope

Area	Work completed
A/B response evaluation	Reviewed three response pairs, applied weighted requirements and critical-failure gates, and checked annotation consistency.
Function calling	Reviewed two supplied recipe workflows and designed two additional examples with parallel operations and conditional fallback.
Enterprise prompt design	Reviewed incident response, calendar booking, and headcount reporting tasks; developed reference tasks and customer-contract annotation guidance.

1. A/B response evaluation

Evaluation method

Each response is assessed independently before a relative preference is assigned.

1. Extract requirements from the system and user prompts.
2. Separate requirements into atomic, observable checks.
3. Fix the weights and identify the small number of critical gates.
4. Score each response against the same requirements.
5. Calculate the weighted score and apply any rating cap.
6. Lock the absolute ratings, then compare the responses.
7. Check that the evidence, comments, scores, and final preference agree.

The independent-before-comparison approach is informed by PRePair. The weights, thresholds, and gates are project-specific calibration choices, rather than a published industry standard.

Requirement weights

Weight	Classification	Meaning
5	Critical	Necessary to achieve the main task; failure materially damages correctness or usefulness.
3	Important	Contributes meaningfully to the request, although an imperfect result can remain useful.
1	Minor	A secondary quality or presentation requirement with limited impact on the core task.

Fulfilment scores

Score	Meaning
4	Fully satisfied
3	Satisfied with a minor defect
2	Partially satisfied, with a meaningful issue
1	Major failure, but some relevant attempt
0	Missing, incorrect, or contradicted

The weighted percentage is:

Weighted score = $100 \times \text{sum}(\text{weight} \times \text{fulfilment score}) / (4 \times \text{sum}(\text{weights}))$

Absolute ratings and gates

Weighted score	Preliminary rating
90-100	Amazing
75-89	Pretty good
55-74	Okay
30-54	Pretty bad
0-29	Horrible

A requirement qualifies as a gate only when it is necessary for the core goal, its failure would make the answer materially wrong or unreliable, other strengths cannot compensate, and the failure can be identified objectively.

- **No gate failure:** retain the score-derived rating.
- **Serious gate failure, scored 0 or 1:** the final rating cannot exceed **Okay**.
- **Fundamental core-task failure:** answering the wrong task, entity, or time period caps the final rating at **Pretty bad**.

These are maximum ratings, not automatic assignments. A lower score-derived rating remains lower.

Relative preference

Apply the following rules in order, using the final absolute ratings after gates:

Condition	Relative preference
Two or more rating bands apart, or one response has a fundamental core-task failure that the other avoids	Much better
Exactly one rating band apart, or the same band with a weighted-score gap greater than 5 percentage points	Better
Same rating band with a gap of 5 percentage points or less	Tie

Results

Sample	Model A	Model B	Final preference
Paris weather prediction	45%, Pretty bad	75%, Pretty good	Model B is much better
Twenty pros and twenty cons of owning a cat	81%, Pretty good	88%, Pretty good	Model B is better
Environmental policy newsletter	82%, Pretty good	68%, Okay	Model A is better

Paris weather prediction

Use case: A user asks whether it will rain in Paris, France, “this weekend.” The model must resolve the weekend against the sample’s fixed current date, retrieve forecast evidence for the correct place and period, and answer the rain question without overstating certainty. The system also requires a matching weather emoji repeated exactly three times at the beginning.

Evaluation: Model A relied primarily on evidence for Paris, Texas instead of Paris, France. This is a fundamental entity-grounding failure. Model B addressed the intended location and supported its conclusion with relevant forecast evidence, although it still missed a formatting requirement.

Cat ownership comparison

Use case: A user asks for 20 pros and 20 cons of owning a cat, presented in a table. The model must provide both sets in the requested quantities, keep the points relevant and meaningfully distinct, and make the comparison easy to read.

Evaluation: Model A supplied 19 pros and 19 cons instead of the requested 20 of each, with some thematic repetition. Model B more fully met the quantity requirement and provided more informative entries. Both remained within the same absolute-rating band.

Environmental policy newsletter

Use case: A user requests an approximately 500-word newsletter draft for an environmental nonprofit, explaining the EU Carbon Border Adjustment Mechanism (CBAM), its 2026 definitive phase, and the implications for French businesses. The request is in English and does not ask for a language change. The evaluation checks policy accuracy, France-specific relevance, length, sourcing, and professional newsletter style.

Evaluation: Model A delivered a more directly usable English-language draft with clearer source transparency. Model B provided deeper France-specific discussion but switched to French, exceeded the requested length, omitted direct source links in the delivered article, and retained editorial residue. The report also identified factual problems in both responses. These are findings from the reviewed sample, not current policy guidance.

Annotation quality findings

The annotation review identified recurring weaknesses:

- Explicit instruction failures were sometimes missed or underweighted.
- Stylistic preferences, such as engagement cues and closing summaries, were treated as requirements without prompt support.
- Factual errors and language mismatches received less attention than surface presentation.
- Absolute ratings did not always reflect the severity of the defects.
- One sample contained contradictory relative-preference selections.

The recommended correction is to link every material criticism to a requirement and observable evidence, then verify that the narrative assessment agrees with the selected ratings.

2. Function-calling evaluation

Two-layer review

The framework separates whether a workflow can execute from how effectively it tests useful tool behavior.

Layer	Check	Evaluation question
Structural validity	Tool availability	Is every called function enabled?
Structural validity	Tool contract validity	Does each call respect the function's actual behavior?
Structural validity	Input availability	Are required arguments available from the prompt or earlier results?
Workflow quality	Tool fit	Do the tools naturally support the request?
Workflow quality	Workflow logic	Are dependencies and branches meaningful?
Workflow quality	Task completeness	Can the workflow satisfy the whole request?
Workflow quality	Tool efficiency	Are unnecessary calls avoided?

Layer	Check	Evaluation question
Workflow quality	QC auditability	Is expected behavior clear enough to grade objectively?

Structural checks are pass/fail. Each of the five quality dimensions receives 0, 1, or 2 points:

- **0:** materially unmet; substantial redesign required.
- **1:** partially met; a material limitation needs targeted revision.
- **2:** fully met; no material limitation requiring revision.

The report interprets 9-10 points with no zero-scored dimension as strong, 6-8 without a major pattern of zero scores as usable with targeted revision, and 0-5 or multiple zero-scored dimensions as weak. Structural failure prevents acceptance as written, regardless of the quality score.

Supplied workflow findings

Sample	Structural validity	Quality score	Decision
Recipes within a time and budget limit, serving four people	Fail	3/10	Requires redesign
Pancakes for six, with a flour substitution and grocery list	Pass	7/10	Requires targeted revision

Sample 1: time, budget, and serving constraints

Use case: A user asks which recipes can be prepared within 30 minutes and a \$15 budget for four people, assuming each ingredient costs about \$3 and the pantry is empty. Only `check_pantry(ingredients, pantry_items)` and `estimate_prep_time(steps)` are enabled. The review asks whether the proposed workflow has the tools and inputs needed to satisfy every constraint.

Evaluation: The proposed workflow required a disabled scaling function. Recipe steps and ingredients also lacked an explicit source because recipe retrieval was unavailable. Serving quantities and cost could not be resolved reliably with the available data. An empty pantry made the pantry check relatively trivial.

Recommended changes:

- Enable recipe retrieval or supply the required recipe data explicitly.
- Define the candidate recipes.
- Remove or redesign serving-quantity requirements unless the tools support them.
- Use a partially populated pantry to create a meaningful comparison.
- Ensure budget calculations have a defined basis.

Sample 2: pancakes, dietary substitution, and shopping needs

Use case: A user wants pancakes for six people, has only butter and sugar, asks for a flour substitute because a guest cannot eat gluten, and requests a grocery list. Recipe retrieval, pantry checking, serving-count scaling, time estimation, and substitution lookup are enabled. The review checks their actual implementations, the dependency order, and whether each call is needed for the request.

Evaluation: The calls used enabled tools with grounded inputs. However, the supplied `scale_recipe()` implementation recorded a serving count without adjusting ingredient quantities, and `check_pantry()` checked ingredient names rather than amounts. The workflow therefore could not produce quantity-aware shopping needs for six people.

The workflow also requested substitutions for ingredients beyond the flour substitution requested by the user. Pantry checking did not need to wait for scaling because that implementation did not modify the ingredient list.

Recommended changes:

- Clarify or remove unsupported quantity requirements.
- Request only the flour substitution.
- Return one clear grocery list showing the substitute and other ingredients to buy.
- Preserve real data dependencies and avoid unnecessary sequencing.

Additional workflow designs

Parallel comparison with conditional substitutions

Task: Determine whether pancakes, stir-fry, or both can be prepared within 50 minutes using only the listed pantry ingredients and valid substitutes already in that pantry.

Expected behavior:

1. Retrieve both recipes independently.
2. For each recipe, estimate preparation time and check pantry contents.
3. Exclude recipes exceeding 50 minutes.
4. For remaining recipes, request substitutes only for missing ingredients.
5. Verify that every proposed substitute is already in the pantry.
6. Return all qualifying recipes and their substitutions, or report that neither works.

The design tests independent calls, shared prerequisites, conditional tool use, and result-dependent validation. The report's design assessment passed all structural checks and awarded 2/2 for each quality dimension.

Preferred recipe with conditional fallback

Task: Try lasagna first. Evaluate stir-fry only if lasagna exceeds 50 minutes or cannot be prepared using pantry ingredients and permitted substitutions.

Illustrative pseudocode:

```

def choose_recipe(pantry_items):
    for recipe_name in ["lasagna", "stir-fry"]:
        recipe = get_recipe(recipe_name)
        prep_time = estimate_prep_time(recipe["steps"])

        if prep_time > 50:
            continue

        missing = check_pantry(recipe["ingredients"], pantry_items)
        substitutions = {}
        feasible = True

        for ingredient in missing:
            substitute = suggest_substitute(ingredient)
            if substitute == "No substitute found" or substitute not in pantry_items:
                feasible = False
                break
            substitutions[ingredient] = substitute

        if feasible:
            return {
                "recipe": recipe["name"],
                "estimated_prep_time": prep_time,
                "substitutions": substitutions,
            }

    return "Neither recipe meets the stated constraints."

```

This is workflow pseudocode using the sample's tool contracts, not a standalone executable application. The fallback runs only after the preferred option fails. The report's design assessment passed all structural checks and awarded 2/2 for each quality dimension.

3. Enterprise prompt and grading review

Enterprise tasks need both realistic work and deterministic expectations. Each review checks the user request, system rules, available evidence, tools, and expected final environment state.

Workflow	Main issue	Recommended correction	Decision
HTTP 503 incident response	Calendar-time interpretation, user identity, and starting incident state were not fully established	Resolve ground truth and make grading checks atomic	Revise
Calendar booking	Meeting duration, relative-date anchor, and no-availability fallback were underdefined	Define the missing scheduling constraints and fallback	Revise
Headcount growth dashboard	Year-over-year growth was confused with cumulative growth	Define the metric, recompute ground truth, and complete output checks	Revise

Incident response

Use case: A customer-facing main application is returning HTTP 503 errors. The model is asked to coordinate the response across Confluence, Slack, Linear, Calendar, Gmail, and Notion: retrieve the backend contact from the runbook, notify the team, create and assign a project, arrange a review meeting, prepare an unsent customer update, and record the incident under the supplied organizational rules.

The workflow tests whether the model can retrieve a backend contact from a runbook, use that evidence in an issue-tracking action, and coordinate notifications, a review meeting, a customer draft, and an incident record.

Grading should verify:

- The correct runbook and backend contact are identified.
- The incident notification reaches the required channel.
- The issue-tracking project has the required title, label, and retrieved owner.
- The review meeting uses the correct date, timezone, attendee, and description.
- The customer email remains a draft and includes the required content.
- Incident fields follow the applicable defaults and existing environment state.
- Returned links and completion statements match actual outcomes.

A sample incident ID in an instruction is an example. The next ID must be derived from the existing incident sequence. Calendar rules and current-user identity must also be established by the environment rather than guessed.

Calendar booking

Use case: A user asks the model to book a meeting for “next Friday” with three named colleagues. The model must identify the participants and current user, inspect all four calendars, find a shared available slot, and create the meeting with the correct organizer and attendees. The sample also raises the case where no common valid slot exists.

The task requires date resolution, participant matching, availability comparison across four calendars, and a correctly organized event.

Missing duration prevents reliable assessment of a valid slot. Relative dates require a fixed environment date. The fallback must address **no common valid slot**, which differs from every participant being individually unavailable.

The revised task should specify the duration, any applicable working-hours and timezone constraints, and the required fallback. Grading should verify correct participants, a common available slot, the organizer and attendee roles, and correct non-booking when no valid slot exists.

Headcount dashboard

Use case: A user asks the model to analyze monthly headcount.csv data for Engineering, Marketing, Sales, HR, and Operations across 2020-2024. The requested deliverables are a department line chart, a stacked area chart, a growth table, and supporting analysis, assembled in a Notion report and shared through #executive-reports. The review examines both analytical correctness and completion of the publishing workflow.

The task combines monthly department trends, a stacked area chart, a growth table, a report page, and distribution through a collaboration channel.

The main defect is a mismatch between the requested year-over-year metric and supplied cumulative-growth percentages. The sample must define both the metric and the aggregation basis before expected values are recomputed.

Additional checks should verify:

- The requested departments and analysis period are used.
- Monthly series are accurate and chronological.
- Each department has a separate layer in the stacked area chart.
- The combined stack represents total headcount.
- The report contains both charts, the growth table, and supporting analysis.
- The report link is shared in the specified channel.
- The title is consistent with the analysis period, or any intentional difference is explained in the task specification.

“Executive-ready” should be translated into observable checks such as readable labels, correct units, clear legends, and accurate data.

4. Reference task designs

Lead prioritization and outreach drafts

This reference task combines a lead spreadsheet with recent email evidence to select three eligible accounts, prepare personalized outreach drafts, summarize the selection, and share the summary.

Eligibility requires at least 500 employees, no active-customer status, and an explicit business need in the lead notes. Priority signals include a request for next steps, buying intent, an unresolved objection, and discussion of the identified need.

The design uses a defined email-evidence window, ranking rules, recency checks, and a final alphabetical tie-break. Each draft must reference supported account evidence, include a next step, and remain unsent.

Grading checks selection, evidence grounding, ranking, correct contacts, draft content, non-sending, summary completeness, and sharing of the created page.

Unresolved incident summary

This reference task selects P0 and P1 incidents within an inclusive reporting window and produces a structured summary.

The rules explicitly define unresolved statuses as **Investigating**, **In Progress**, and **Monitoring**. **Resolved** and **Closed** incidents are excluded.

The output includes incident ID, description, priority, status, owner, and creation date. Incidents are sorted by creation date from oldest to newest, then by incident ID in ascending order when dates match. Grading verifies both inclusion of qualifying incidents and exclusion of non-qualifying incidents.

5. Customer-contract annotation guideline

Purpose and scope

Create realistic contract-workflow tasks that are supported by the environment and consistently gradable. Applicable scenarios include renewals, expiry and notice deadlines, owner identification, prioritization, conditional follow-up, and structured reporting.

Exclude unsupported legal conclusions, undefined risk labels, unnecessary tools, and tasks whose required evidence is unavailable.

Sample creation process

1. **Define the target population.** Specify exactly which contracts or accounts are in scope.
2. **Define evidence and decision rules.** Use observable facts such as dates, status, notice periods, support issues, and explicit customer concerns.
3. **Define prioritization and tie-breaks.** Ensure multiple qualifying candidates can be ranked deterministically.
4. **Define outputs and conditional actions.** State what must be produced and when follow-up must or must not occur.
5. **Verify the environment.** Confirm that evidence, ownership information, tool support, and fixed temporal context are available and mutually consistent.
6. **Write atomic grading criteria.** Each criterion checks one meaningful outcome grounded in instructions or environment evidence.

Missing information is not itself evidence of risk unless the task explicitly defines that rule. Grade required outcomes rather than a single navigation path unless that path is intentionally being tested.

Acceptance criteria

- The target population can be determined exactly.
- Required evidence and ownership information are available.
- Decision rules and priority order are explicit.
- Tie-breaks resolve otherwise ambiguous selections.
- Conditional actions have clear triggers.
- Criteria cover every material requirement, including correct non-action.
- Time-dependent rules use a fixed environment date, time, and timezone where relevant.
- Two annotators using the same environment should reach the same required decisions and final state.

Final decision

Decision	Meaning
Accept	Workflow, evidence, business rules, environment, and grading criteria are complete and deterministic.
Revise	The use case is valid, but rules, environment requirements, conditional actions, or grading criteria need clarification.
Reject	The workflow is unrealistic, unsupported by the environment, or cannot be graded reliably.

Key takeaways

- A polished answer can still fail its core task.
- Tool availability and input provenance must be checked before judging workflow quality.
- More tool calls, longer responses, and more sources do not independently establish quality.
- A valid tool call does not prove that the overall user request can be completed.
- Deterministic evaluation requires alignment between the prompt, environment, business rules, and grading criteria.
- Correct non-action is part of successful task completion.

References

References retained from the report for the evaluation approach and general tool-use concepts:

- Jeong, H., Park, C., Hong, J., Lee, H., & Choo, J. (2025). The Comparative Trap: Pairwise Comparisons Amplifies Biased Preferences of LLM Evaluators. Proceedings of the 8th BlackboxNLP Workshop, 79-108. [Paper](#)
- Tool use with Claude. [Documentation](#)
- Function calling. [API documentation](#)